

Comparative Analysis of Modern TCP Congestion Control Algorithms

Nicholas Carpenedo, Allison Dana, Rohin Shanker
University of California, Berkeley
Electrical Engineering 122
May 10, 2026

INTRODUCTION

Problem Statement

The goal of this project is to evaluate and compare the performance of modern TCP congestion control algorithms under a range of emulated network conditions. While traditional congestion control schemes like CUBIC are widely deployed, they are known to be sensitive to high latency and packet loss, which are common in environments such as satellite communication links. Our research focuses on understanding how each algorithm behaves under controlled variations in delay, packet loss, and link characteristics, with particular emphasis on scenarios representative of satellite-based communication systems to determine the optimal algorithm for such network conditions.

Background

BBRv3

Unlike loss-based CCAs like Reno and CUBIC, BBR builds an explicit model of the network path. BBR operates by periodically probing the bottleneck bandwidth and round trip time of the link, and sends just enough data to fully utilize the connection without overloading it. Because it is a model based algorithm, it is largely resistant to random packet loss, and it minimizes queueing delay by not filling buffers at the bottleneck.

Reno

Reno controls congestion by modulating how much unacknowledged data the sender can have in flight via a congestion window. The algorithm has two primary operating modes: slow start and congestion avoidance. During slow start, the congestion window is multiplicatively increased until it reaches the slow start threshold. Next, it enters congestion avoidance, where the window increases linearly (~one segment per RTT). Upon a timeout-based loss, the congestion window collapses back to one segment, and triple-duplicate acknowledgments trigger fast retransmit and

recovery. Reno can underutilize high-bandwidth-delay-products or lossy paths given that it slowly increases its congestion window after a reduction.

CUBIC

Cubic is also a loss-based congestion control algorithm, primarily designed for high-speed long-distance networks. Unlike Reno, CUBIC uses a cubic window-growth function rather than a linear one. When a congestion event occurs, the congestion window is multiplicatively reduced. After this, the window grows rapidly, slowing down around the previous loss point, and then accelerates if the network does not experience another loss event. This allows bandwidth to be recovered more efficiently than Reno on high-bandwidth-delay-product paths, however the algorithm is still sensitive to random loss that it interprets as congestion.

Vegas

Vegas is a delay-based congestion control algorithm that attempts to proactively detect congestion before packet loss occurs. The algorithm estimates the path's base RTT by comparing the minimum RTT observed over the connection with the expected throughput. This is computed from the current congestion window and base RTT, to the actual throughput measured from acknowledgments. If the actual throughput is close to the expected throughput, Vegas increases its congestion window. If the actual throughput falls far enough below the expected throughput, the window gets decreased under the assumption that queues are building in the network. Vegas's approach can avoid some packet loss and reduce queueing delay, however this depends on accurate RTT measurements and may become conservative under jitter, variable delay, or competition with more aggressive loss-based algorithms.

Related Work

Prior work has studied BBRv3 as a response to limitations in earlier BBR versions, particularly fairness, coexistence with loss-based algorithms, and behavior across heterogeneous network paths. *Promises and Potential of BBRv3*¹ provides a broad empirical evaluation of BBRv3 across network scenarios including different buffer sizes, round-trip times, packet loss rates, and flow-size distributions. The paper finds that BBRv3 improves convergence behavior relative to earlier BBR versions in some cases, while still identifying open issues such as degraded convergence for staggered flows, long-RTT bias, and poor coexistence with CUBIC. This work motivates our focus on BBRv3, but our study differs by isolating satellite-like link profiles with combined delay, jitter, loss, and bandwidth constraints representative of terrestrial, LEO, and GEO communication paths.

Other studies have focused more specifically on BBRv3 performance across paths with different RTTs and buffer sizes. For example, *Performance Evaluation of TCP BBRv3 in Networks with Multiple Round Trip Times*² evaluates BBRv3 under varying RTT and buffer configurations and

considers whether BBRv3 improves fairness relative to earlier BBR versions and loss-based congestion control. This is directly relevant to our project because satellite links are characterized by high RTT and nontrivial buffering effects. However, our work differs by comparing BBRv3 against additional TCP congestion control baselines, including CUBIC, Reno, and Vegas, under link profiles intended to approximate terrestrial, LEO satellite, and GEO satellite conditions.

RFC 9743³ provides guidance for evaluating congestion control algorithms, emphasizing that new or alternative congestion control mechanisms should be tested for efficiency, stability, fairness, and safe coexistence with existing Internet traffic. This aligns with our methodology by encouraging controlled experiments across multiple impairment dimensions rather than evaluating performance under only one idealized condition. In our tests, we vary delay, packet loss, jitter, and link rate while measuring throughput across repeated iperf3 trials. Queue depth is held at $2 \times \text{BDP}$ in the current experiments, with additional buffer-depth testing left as future work.

Overall, existing work shows that BBRv3 is promising, especially because it models bottleneck bandwidth and RTT rather than relying primarily on packet loss. However, less attention has been given to a direct side-by-side comparison of BBRv3, CUBIC, Reno, and Vegas under systematically emulated satellite-style conditions. Our project addresses this gap by using controlled tc/netem experiments to evaluate how these algorithms respond to high delay, packet loss, jitter, and constrained link rates.

METHODS

Environment Setup

We emulate a realistic network environment using two virtual machines. Each virtual machine is running Ubuntu 25.10 for ARM, and is allocated 2 CPU cores, 4GB of RAM, and 20GB disk. The VMs are configured as linked clones to save space and maintain comparable behavior. Both virtual machines are running on the same machine, which is an M1 MacBook Pro. Their virtual network interfaces are set to “Host Only,” which creates an isolated network with only the host and the VMs to minimize traffic from other devices. Additionally, we have configured the VMs to use VMXNET3 virtual network interface, which provides higher performance and more realistic behavior compared to the default e1000, and NIC rates are limited to 100Mbps to better replicate the limitations of a real device.

We intentionally keep modifications to the default operating system / network stack minimal. One critical adjustment we make is that we dramatically increase the interface queue depths and TCP memory limits so that the only bottleneck is presented by the emulated link. We also disable

hardware offloading (TSO, GSO, LRO, GRO) to maintain accurate and consistent behavior while testing on virtual machines.

To emulate network behavior, we use the Linux utility `tc` (traffic control), which allows configuration of queueing disciplines (qdiscs) within the kernel networking stack. In particular, we use the `netem` (network emulator) qdisc to introduce controlled network impairments such as delay, packet loss, and jitter before packets are processed by higher-level networking components.

To ensure accurate emulation of end-to-end network conditions, we apply `netem` on the receiving side of the communication path. This avoids interference with sender-side TCP behavior, such as TCP Small Queues or TSQ, and allows more controlled modeling of network path characteristics.

Since Linux cannot directly apply qdiscs to ingress traffic, we use an Intermediate Functional Block (IFB) device to redirect incoming packets. In this setup, ingress traffic from the virtual network interface is mirrored to the IFB device, where the `netem` qdisc is applied. After processing, packets continue up the networking stack as if they had experienced the emulated conditions on the network path.

Procedure

For each test, we generate traffic using `iperf3`, which will create a TCP flow between the client and server, allowing us to measure the throughput of a particular TCP CCA. We run `iperf` in reverse mode so that the data originates from the server, allowing us to install the `netem` qdisc on the client machine. Additionally, at high latency, major burstiness (likely due to kernel buffering or virtualization) is observed in the `iperf3` log on the sender, so we record the results on the receiver side instead.

Each test is run for a duration of 180 seconds, with an additional 60 second “warm up” period for which the data is omitted from the final average. For each set of test conditions, we run 5 tests and plot the average result of the five runs. This is repeated for all algorithms, which are CUBIC, RENO, Vegas, and BBRv3.

All setup and automated testing scripts can be found on our project repository:

<https://github.com/rohinshanker/EE-122-simulation>

Note on Queue Depth

All tests below are listed with a queue depth of $2 \times \text{BDP}$, which is considered a very deep buffer. We expect that this will favor loss-based CCAs. Future work will include testing in a variety of buffer conditions, such as $1 \times \text{BDP}$ and $\frac{1}{2} \times \text{BDP}$ conditions.

Below are the calculations for how we determined the queue depth for each test case.

$$\text{Limit} = 2 \times \text{BDP} = 2 \times \frac{\text{Bottleneck Bandwidth} \times \text{RTT}}{8 \times \text{Packet Size (Mb)}}$$

$$\text{MSS} = 1500 \text{ Mb}$$

So for an example test with 300ms delay and 50mbit rate:

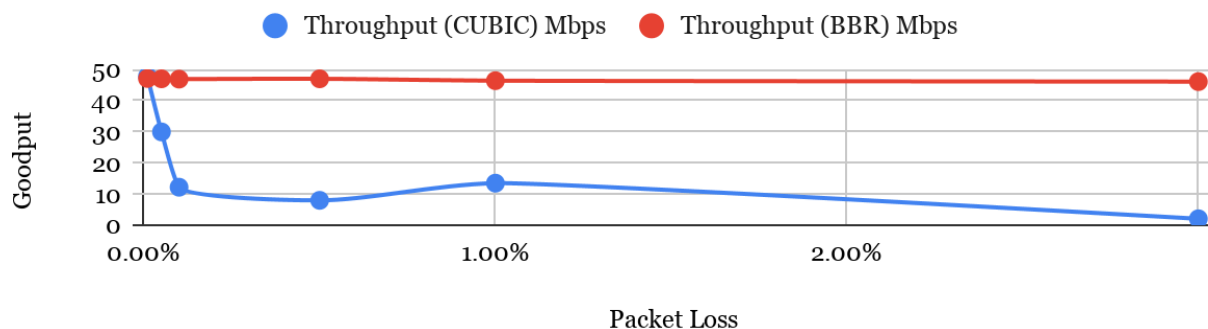
$$\text{Limit} = 2 \times \frac{50\text{Mbps} \times 300\text{ms}}{8 \times 1500\text{b}} = 2 \times \frac{50,000,000 \times 0.3}{8 \times 1500} = 2500 \text{ packets}$$

RESULTS

Preliminary Data (Pre-Poster Presentation)

This section outlines our preliminary results using data from limited trials that we conducted before the in-class poster presentation. Note that only a single trial was performed per test case, so interpret this data with some skepticism. Our final results in the following section were collected using the rigorous methodology described above, and largely validate these conclusions.

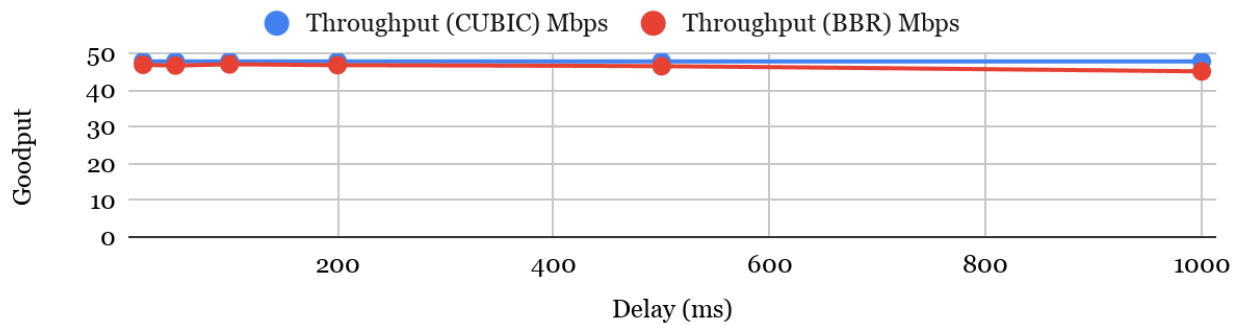
Throughput (CUBIC) Mbps and Throughput (BBR) Mbps



To evaluate the impact of packet loss on throughput and stability, we varied the packet loss rate while keeping the network delay constant. The experiments were conducted with a fixed delay of 50 ms, 0 ms jitter, a 50 Mbps link rate, and a queue depth of $2 \times \text{BDP}$.

Under these conditions, BBRv3 maintained a high and stable throughput of approximately 45 Mbps across all loss rates. In contrast, CUBIC throughput decreased significantly as packet loss increased, indicating greater sensitivity to lossy network conditions.

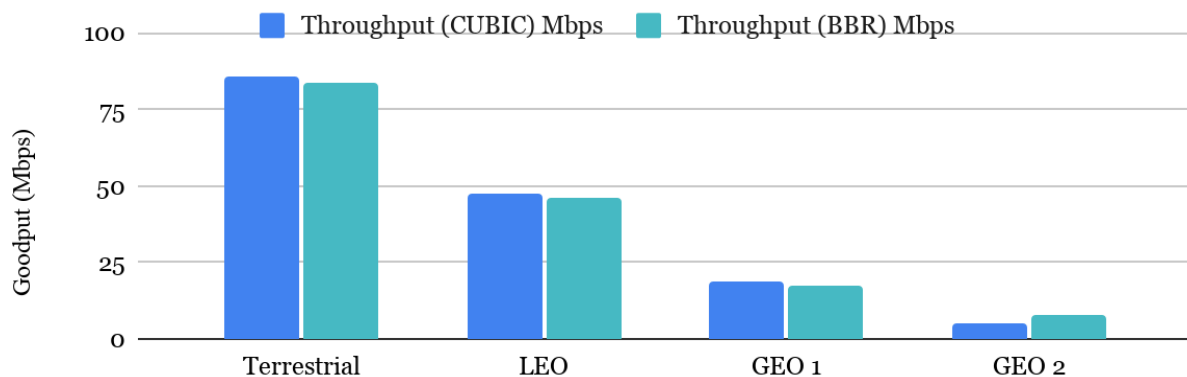
Throughput (CUBIC) Mbps and Throughput (BBR) Mbps



To evaluate performance under increasing RTT, we varied the network delay while maintaining zero packet loss. The experiments were conducted with 0 ms jitter, a 50 Mbps link rate, and a queue depth of $2 \times \text{BDP}$.

Under these conditions, BBR slightly outperformed CUBIC across the tested delay values. We hypothesize that BBR would demonstrate a larger performance advantage under shallow buffer conditions, where its bandwidth and RTT estimation mechanisms may better utilize available network capacity.

Throughput vs Link Type



To evaluate performance under realistic networking environments, we simulated representative terrestrial, Low Earth Orbit (LEO) satellite, and Geostationary Earth Orbit (GEO) satellite link conditions.

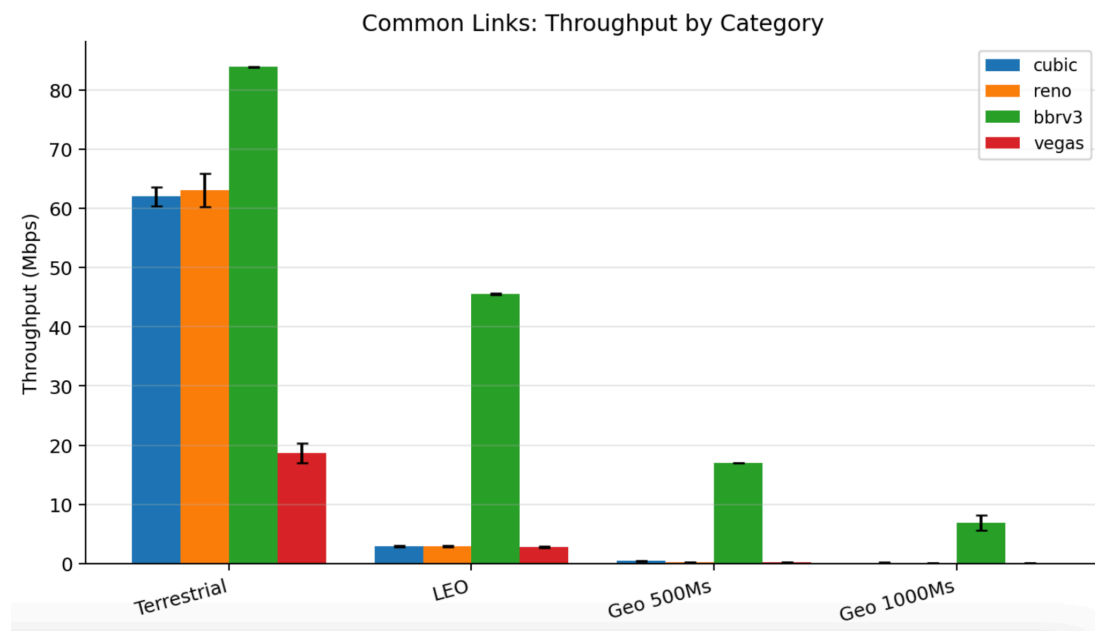
All experiments were conducted with a packet loss correlation of 25% and a queue depth of $2 \times \text{BDP}$. Additional parameters for each category are listed in the table below.

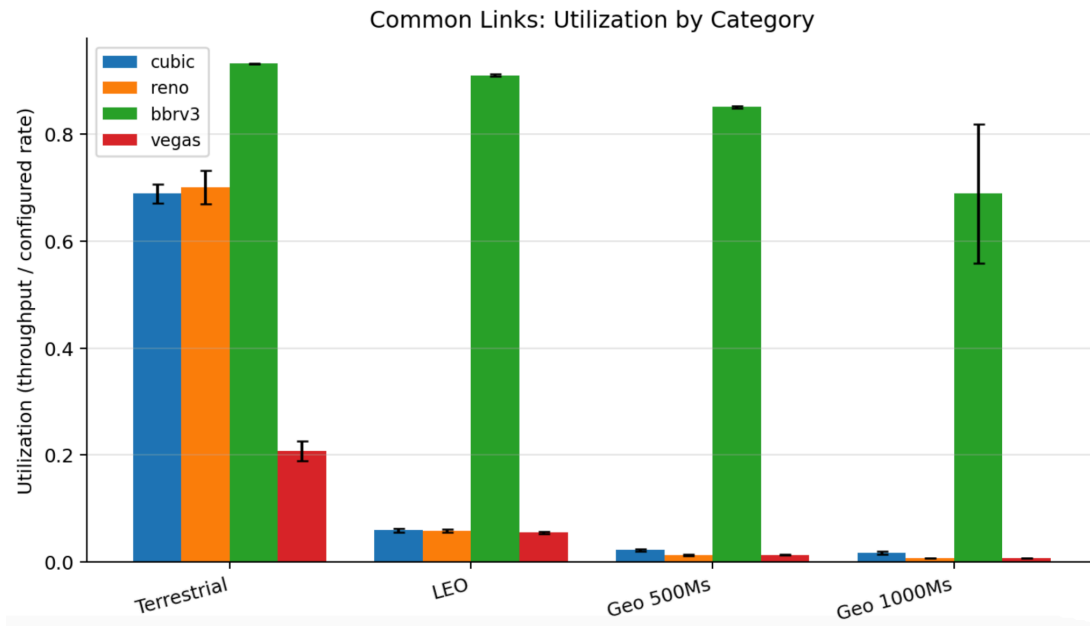
Category	Delay (ms)	Jitter (ms)	Rate (Mbps)	Loss
Terrestrial	20	2	90	0.01%
LEO	80	8	50	0.30%
GEO 1	500	50	20	1%
GEO 2	1000	100	10	3%

Both algorithms perform comparably, with CUBIC slightly outperforming BBRv3 for the Terrestrial, LEO, and GEO 1 test cases. BBRv3 significantly outperformed Cubic in the GEO 2 case, likely due to the high packet loss condition.

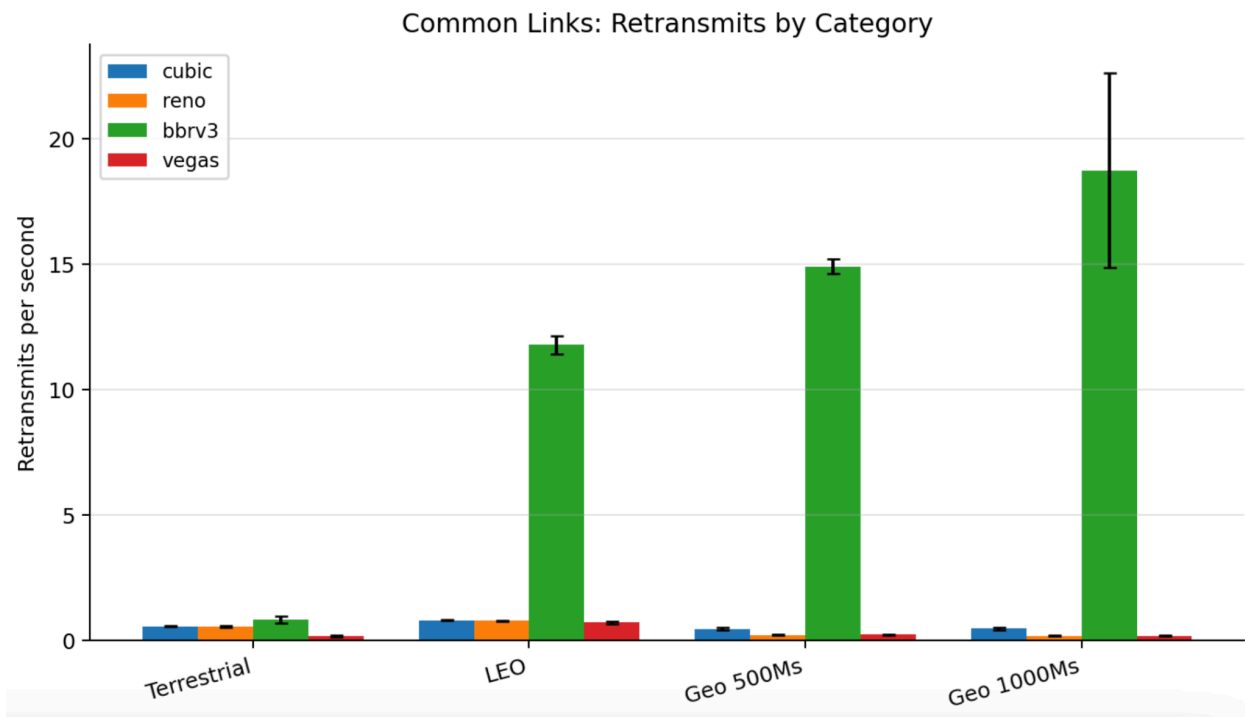
Post-Presentation Trials

After collecting all data for Cubic, BBRv3, Reno and Vegas algorithms, and plotting the results, we see that BBR continues to outperform in the throughput and utilization throughput categories.

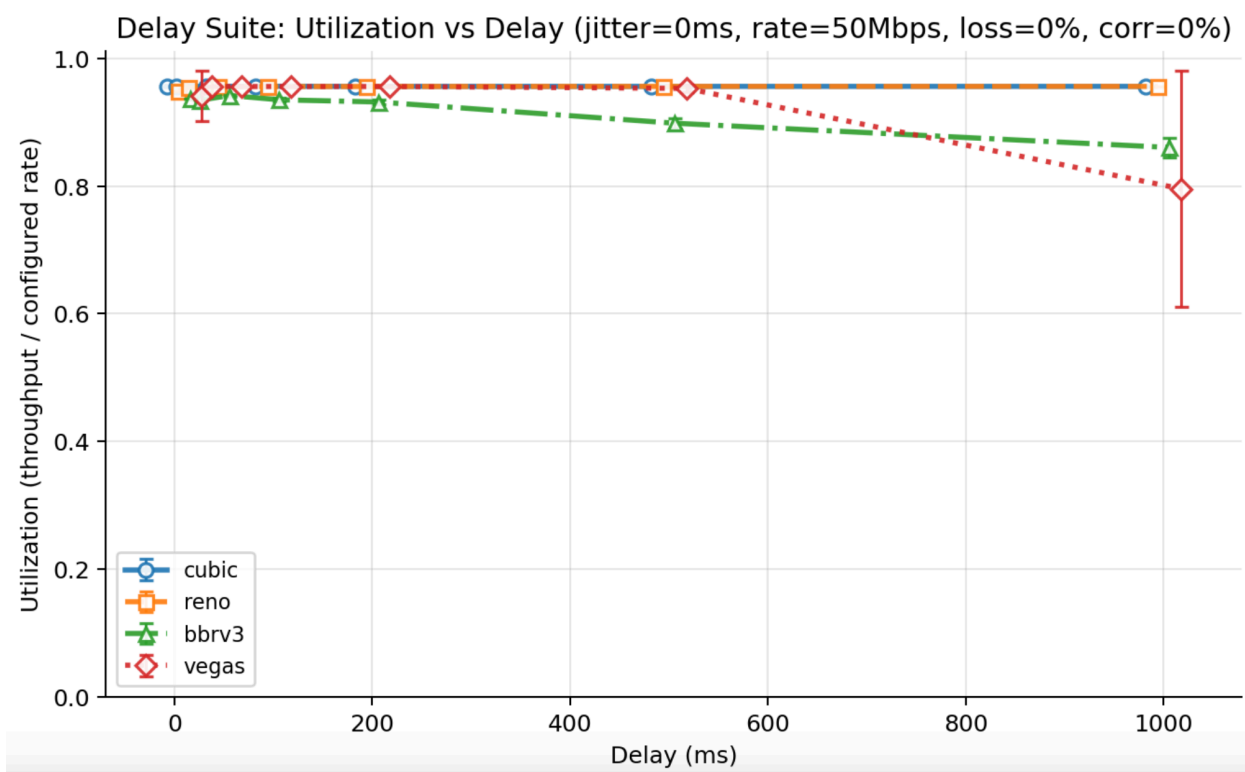
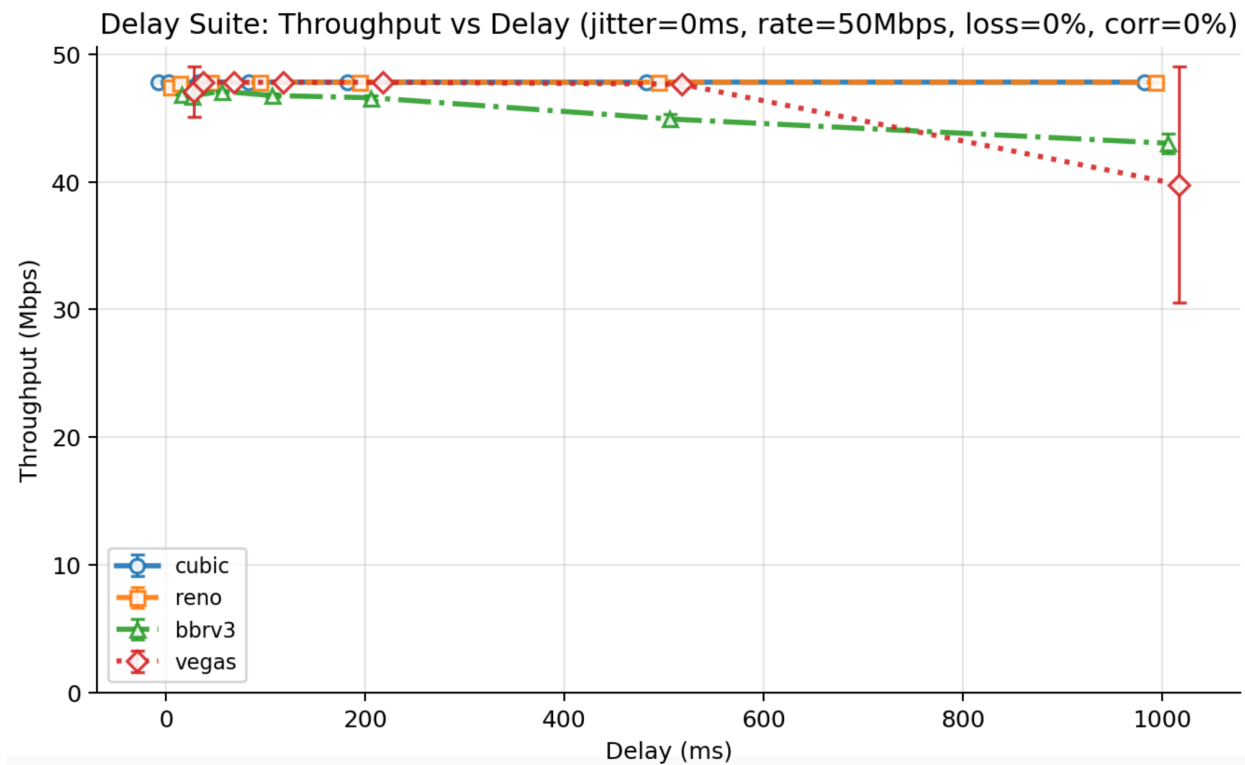




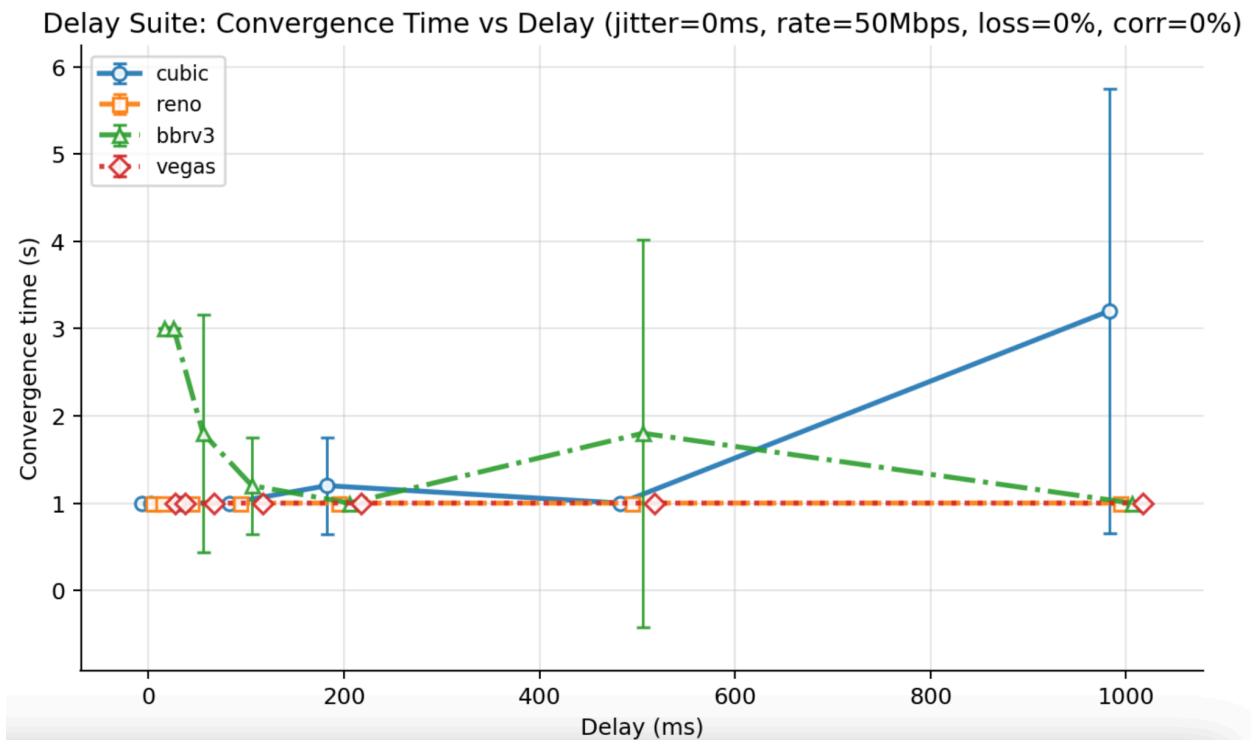
However, there is a bigger picture: we notice that BBRv3 is the highest by a large factor when it comes to retransmission numbers. While BBRv3 is designed to maximize throughput through aggressive bandwidth probing, these results suggest that on GEO networks it may introduce instability or excess packet loss, whereas Reno, CUBIC, and Vegas remain comparatively stable.



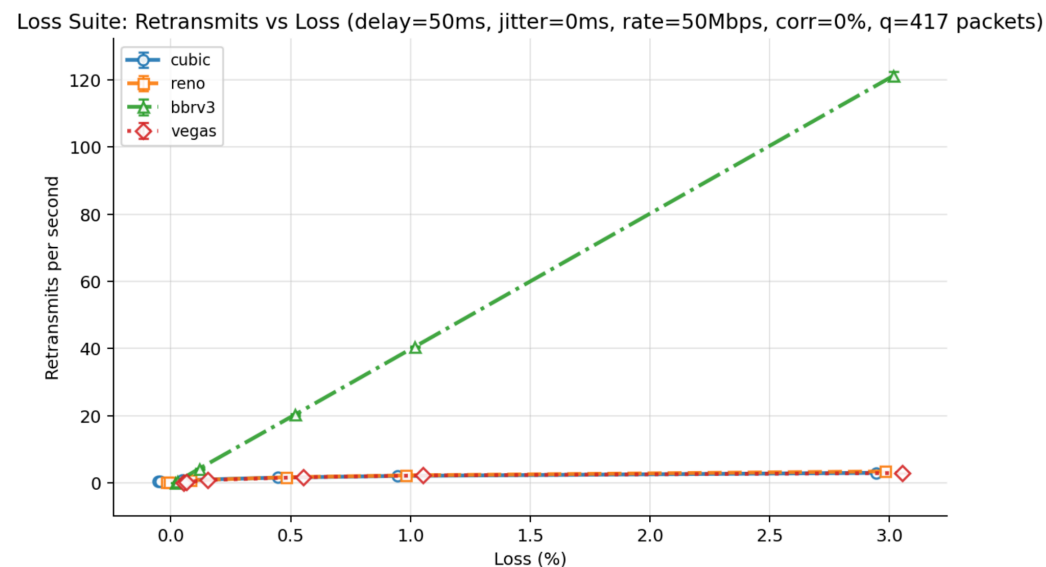
Looking at delay vs throughput, TCP Cubic and Reno are the most stable, with BBRv3 consistently tapering off and Vegas experiencing the highest drop at GEO-scale delays.



Reno and Vegas maintained fast convergence times across all RTTs, while CUBIC showed increased convergence time at very high delays despite preserving strong throughput performance. BBRv3 exhibited the most inconsistent convergence behavior. We can see this in the pretty significant fluctuations across delay periods.

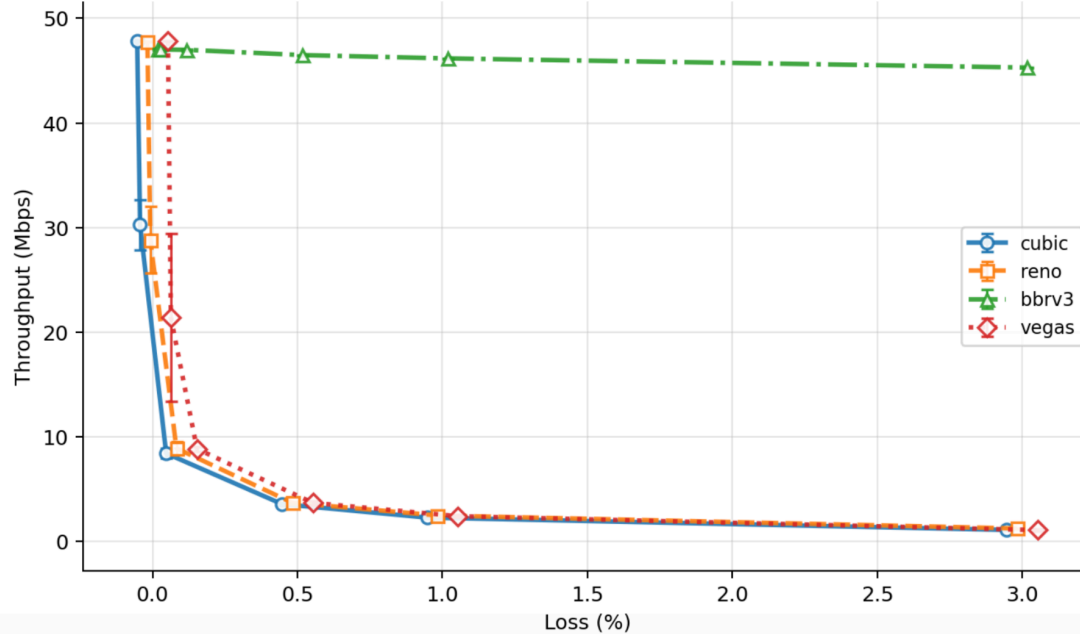


Regarding loss, in accordance with earlier conclusions, BBRv3 is the highest in retransmission rates given percent loss. This is likely because its congestion control strategy is much more aggressive than the other algorithms, consistently transmitting at higher rates.

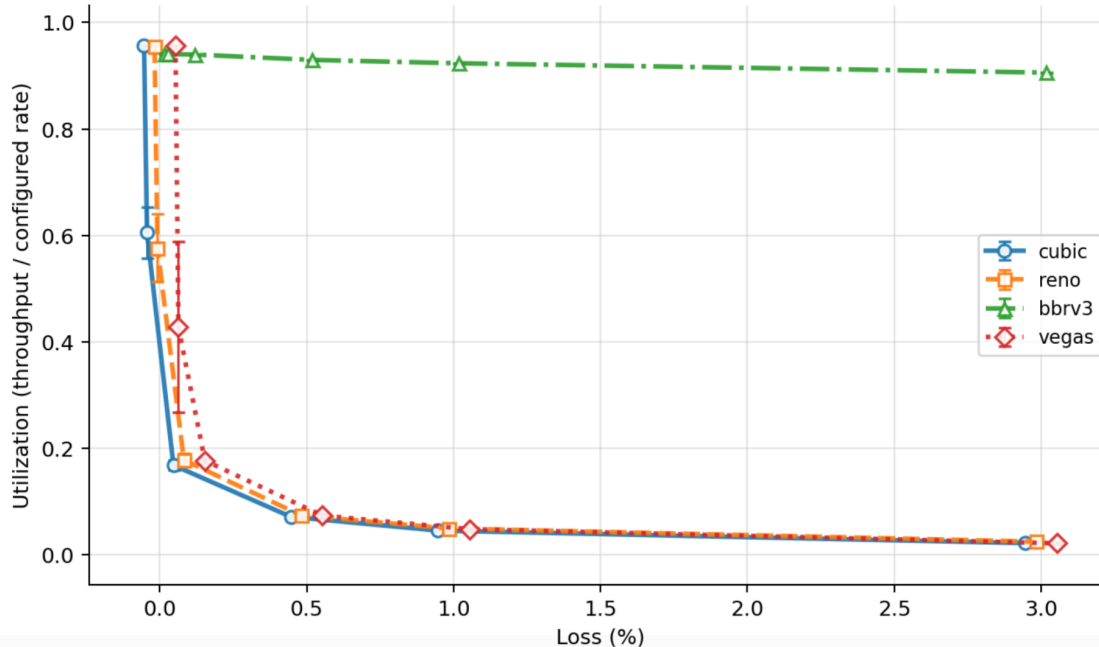


To compensate for the more aggressive algorithm when it comes to retransmission, we can see that the throughput and utilization rates observed are much higher in BBRv3 than the other algorithms, as before.

Loss Suite: Throughput vs Loss (delay=50ms, jitter=0ms, rate=50Mbps, corr=0%, q=417 packets)



Loss Suite: Utilization vs Loss (delay=50ms, jitter=0ms, rate=50Mbps, corr=0%, q=417 packets)

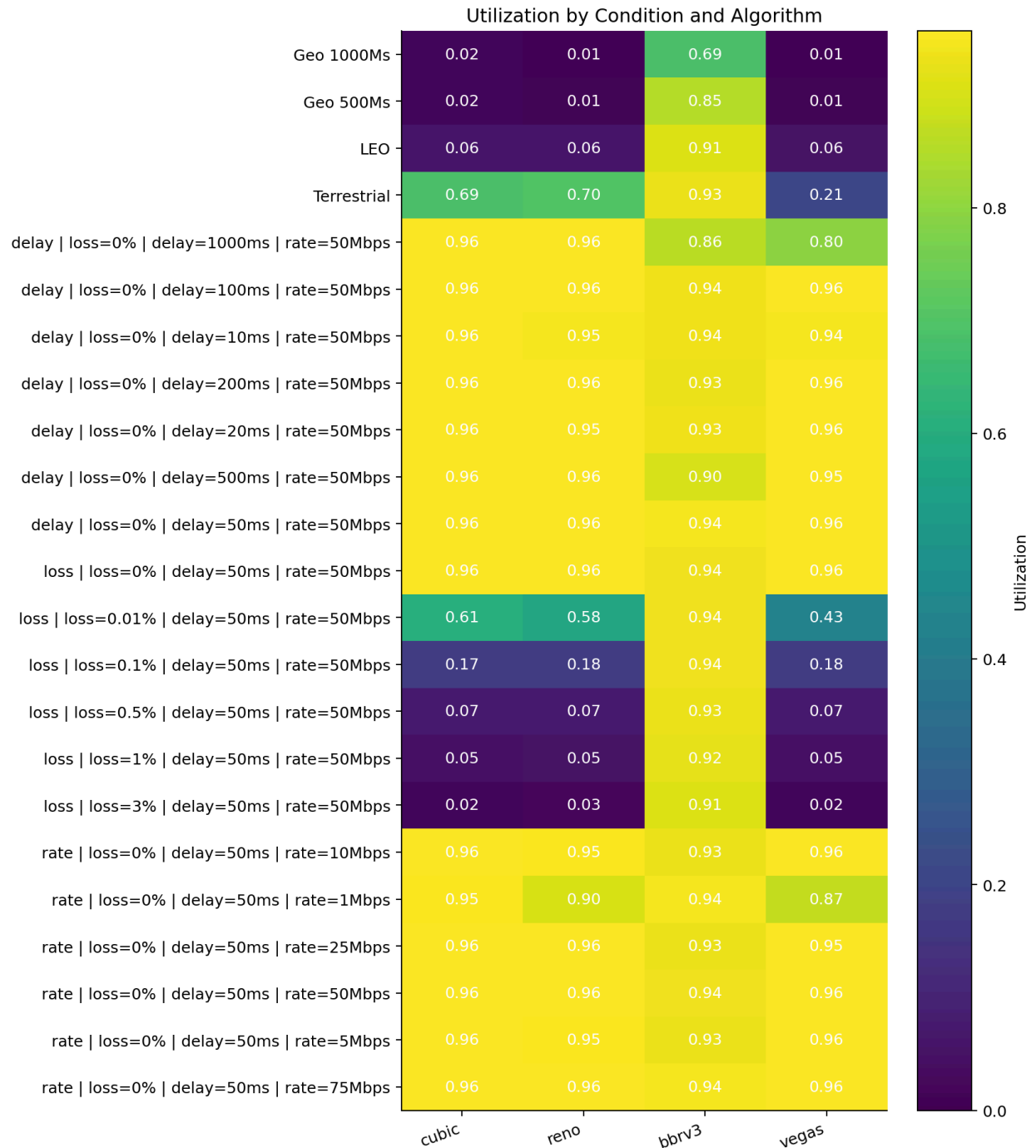


There are more specific test case graphs available for viewing on the GitHub to visualize specific conditions, however there is a “winner table” of specific conditions outlined below of algorithm “bests” and which algorithm is most effective in each category. The competition is clearly between BBRv3 and Cubic:

Winner Table by Condition

Condition	Best Throughput	Mbps	Best Utilization	Util.
Terrestrial	bbrv3	83.88	bbrv3	0.932
LEO	bbrv3	45.52	bbrv3	0.910
Geo 500Ms	bbrv3	17.02	bbrv3	0.851
Geo 1000Ms	bbrv3	6.89	bbrv3	0.689
delay loss=0% delay=10ms rate=50Mbps	cubic	47.82	cubic	0.956
delay loss=0% delay=20ms rate=50Mbps	cubic	47.82	cubic	0.956
delay loss=0% delay=50ms rate=50Mbps	cubic	47.82	cubic	0.956
delay loss=0% delay=100ms rate=50Mbps	cubic	47.83	cubic	0.957
delay loss=0% delay=200ms rate=50Mbps	cubic	47.82	cubic	0.956
delay loss=0% delay=500ms rate=50Mbps	cubic	47.82	cubic	0.956
delay loss=0% delay=1000ms rate=50Mbps	cubic	47.82	cubic	0.956
loss loss=0% delay=50ms rate=50Mbps	cubic	47.82	cubic	0.956
loss loss=0.01% delay=50ms rate=50Mbps	bbrv3	47.03	bbrv3	0.941
loss loss=0.1% delay=50ms rate=50Mbps	bbrv3	46.99	bbrv3	0.940
loss loss=0.5% delay=50ms rate=50Mbps	bbrv3	46.49	bbrv3	0.930
loss loss=1% delay=50ms rate=50Mbps	bbrv3	46.17	bbrv3	0.923
loss loss=3% delay=50ms rate=50Mbps	bbrv3	45.30	bbrv3	0.906
rate loss=0% delay=50ms rate=1Mbps	cubic	0.95	cubic	0.950
rate loss=0% delay=50ms rate=5Mbps	cubic	4.78	cubic	0.957
rate loss=0% delay=50ms rate=10Mbps	cubic	9.56	cubic	0.956
rate loss=0% delay=50ms rate=25Mbps	cubic	23.91	cubic	0.956
rate loss=0% delay=50ms rate=50Mbps	cubic	47.82	cubic	0.956
rate loss=0% delay=50ms rate=75Mbps	cubic	71.73	cubic	0.956

We also have a heat map mapping utilization rates across conditions and algorithms. Visually, it is clear that BBRv3 outperforms the other algorithms in this capacity. The yellow is max utilization, and the purple is the min. BBRv3 is majority yellow and green across all conditions, and more yellow across most indicating a higher overall efficiency.



CONCLUSION

Overall, while performance is comparable between algorithms in excellent network conditions, BBRv3 outperforms the other algorithms in poor link conditions including high packet loss and low latency. CUBIC surprisingly outperformed BBRv3 in the delay test, though we hypothesize that BBR would demonstrate a larger performance advantage under shallow buffer conditions,

where its bandwidth and RTT estimation mechanisms may better utilize available network capacity. BBRv3 is the clear choice for high loss, high latency networks like GEO satellite links.

FUTURE WORK

With additional time, there are a variety of improvements and additional cases to investigate. Primarily, we plan on testing a variety of buffer depths. All of our data was collected with $2 \times$ BDP queue depth, but this is not necessarily representative of all networks (especially satellite links) and may be overly generous to loss-based algorithms. Testing with shallower buffers may explain the unexpected result of CUBIC outperforming BBRv3 in the delay testing.

Additionally, we would like to measure latency as a performance metric in addition to bandwidth. Further investigation with competing parallel flows and fairness between algorithms would be informative.

ACKNOWLEDGEMENTS

This project was developed in collaboration with LabN, and we would like to thank Dan Lukaszewski for his support and guidance throughout the project.

REFERENCES

1. Zeynali, D., Weyulu, E. N., Fathalli, S., Chandrasekaran, B., & Feldmann, A. (2024). *Promises and potential of BBRv3*. In P. Richter, V. Bajpai, & E. Carisimo (Eds.), *Passive and Active Measurement: 25th International Conference, PAM 2024, Virtual Event, March 11–13, 2024, Proceedings, Part II* (Lecture Notes in Computer Science, Vol. 14538, pp. 249–272). Springer. https://doi.org/10.1007/978-3-031-56252-5_12
2. Piotrowska, A. (2024). Performance evaluation of TCP BBRv3 in networks with multiple round trip times. *Applied Sciences*, 14(12), Article 5053. <https://doi.org/10.3390/app14125053>
3. Duke, M., & Fairhurst, G. (2025). *Specifying new congestion control algorithms* (RFC 9743). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9743>